Ölümcül Kilitlenmeler (Deadlocks) - Sınav Hazırlık Özeti (Bölüm 2)

Daha önceki süreç senkronizasyonu konularının devamı niteliğindeki en kritik sınav konusudur. İşletim sistemlerinde proseslerin kaynakları (yazıcı, dosya, bellek vb.) paylaşırken düştükleri çıkmaz sokağı inceler.

---

1. Deadlock (Ölümcül Kilitlenme) Nedir?

İki veya daha fazla prosesin, sadece diğer bekleyen prosesler tarafından gerçekleştirilebilecek bir olayı (kaynağın serbest bırakılmasını) sonsuza kadar beklemesi durumudur.

> 💡 ***Gerçek Hayat Benzetmesi:*** *Tek şeritli dar bir köprüde iki arabanın kafa kafaya gelmesi. İkisi de diğerinin geri gitmesini bekler, kimse geri gitmez ve trafik sonsuza kadar kilitlenir.*

---

2. 🚨 BANKO SINAV SORUSU: Deadlock Oluşması İçin Gereken 4 Şart

Hocaların **EN SEVDİĞİ** klasik sınav sorusudur. Deadlock oluşması için aşağıdaki **4 şartın aynı anda (eşzamanlı) gerçekleşmesi ZORUNLUDUR.** (Coffman Şartları olarak da bilinir). Biri bile bozulursa kilitlenme olmaz.

Şartın Adı (İngilizce - Türkçe)	Ne Anlama Geliyor?	Banka / Restoran Benzetmesi
1. Mutual Exclusion (Karşılıklı Dışlama)	Kaynak aynı anda sadece tek bir proses tarafından kullanılabilir (Paylaşılamaz kaynak).	Bir tuvalet kabinini aynı anda sadece bir kişi kullanabilir.
2. Hold and Wait (Tut ve Bekle)	Bir proses en az bir kaynağı elinde tutarken, başkasındaki diğer kaynakları da almak için bekler.	Elindeki çorbayı tutarken, başkasının elindeki tuzu beklemek.
3. No Preemption (Zorla Geri Alamama)	Bir kaynağı elinde tutan prosesten o kaynak zorla alınamaz. Kendi isteğiyle bırakmalıdır.	Masada yemek yiyen adamın elinden çatalı zorla çekip alamamak.
4. Circular Wait (Döngüsel Bekleme)	Bekleyen prosesler zinciri döngü oluşturur. P0 -> P1'i, P1 -> P2'yi, P2 -> P0'ı bekler.	A, B'nin parasını; B, C'nin parasını; C de A'nın parasını bekler. Herkes kilitlenir.

> ❓ ***Hocanın Tuzağı:*** *"Bu dört şarttan sadece 3 tanesi gerçekleşirse deadlock olur mu?"* ✅ ***Cevap:*** *Hayır! Dördü **aynı anda** (simultaneously) gerçekleşmek zorundadır.*

---

3. Kaynak Tahsis Grafiği (Resource-Allocation Graph)

Proseslerin ve kaynakların durumunu oklarla gösteren grafiklerdir.

- **Daire (Yuvarlak):** Prosesleri temsil eder (P1, P2).
- **Kare/Dikdörtgen:** Kaynakları temsil eder (R1, R2). İçindeki noktalar, kaynağın kaç tane (instance) olduğunu gösterir.

🚩 Grafiğe Bakarak Kilitlenme Tespiti:

- Grafikte **DÖNGÜ (Cycle) YOKSA**, kesinlikle kilitlenme (deadlock) **YOKTUR.**
- Grafikte **DÖNGÜ VARSA**:
  - Eğer her kaynağın **SADECE BİR TANE** kopyası (instance) varsa -> Kesinlikle **DEADLOCK VARDIR.**
  - Eğer kaynakların **BİRDEN FAZLA** kopyası varsa -> Deadlock **OLABİLİR de OLMAYABİLİR de** (Kesinlik yoktur, ihtimaldir).

---

4. Deadlock ile Başa Çıkma Yöntemleri (Karşılaştırma Tablosu)

İşletim sistemlerinin kilitlenmelere karşı 4 temel stratejisi vardır. Karşılaştırma olarak sormayı çok severler.

Yöntem	Stratejisi	Mantığı / Açıklaması
1. Prevention (Önleme)	Kuralları Katılaştır	Baştan o 4 şartın (Mutual Exclusion, Hold and Wait vb.) *en az birinin* asla gerçekleşmemesini sağlayacak katı kurallar koyar. Cihaz verimini çok düşürür.
2. Avoidance (Kaçınma)	İleri Görüşlülük	Sistem kaynak talep edildiğinde geleceği hesaplar. Eğer bu talep bizi güvensiz bir duruma (Unsafe State) sokacaksa talebi şimdilik reddeder / bekletir. *(Bkz. Banker Algoritması)*
3. Detection & Recovery (Tespit ve Kurtarma)	Serbest Bırak	Sistem başta hiçbir şeye karışmaz. Kilitlenme olmuştur der ve periyodik olarak kontrol eder. Kilitlenme bulursa proseslerden birini öldürerek (kill) düğümü çözer.
4. Ostrich Algorithm (Devekuşu Alg.)	Görmezden Gelme	Kafayı kuma gömmektir. **Windows ve Linux'un kullandığı yöntemdir!** Çünkü deadlock çok nadir olur, bunu sürekli kontrol etmek müthiş bir performans kaybıdır. Olursa kullanıcı sistemi resetler (yeniden başlatır).

---

5. Bankacı Algoritması (Banker's Algorithm) - (Avoidance)

Deadlock "Kaçınma" stratejisinin kalbidir. Dijkstra tarafından geliştirilmiştir. Tıpkı bir bankerin müşterilerine kredi verirken bankayı batırmayacak bir sıra bulmasına benzer.

- **Mantık:** Sistem, proseslere kaynak vermeden önce "Eğer bunu verirsem, kalan kaynaklarla herkesin işini bitirmesini sağlayacak **GÜVENLİ BİR SIRA (Safe Sequence)** bulabilir miyim?" diye bakar. Bulursa verir, bulamazsa bekletir.
- **Safe State (Güvenli Durum):** En azından bir tane problemsiz çalışma sırası bulunabilen durumdur. **Güvenli durumdayken ASLA deadlock olmaz.**
- **Unsafe State (Güvensiz Durum):** Deadlock olma **ihtimalinin** (kesin değil, ihtimal) doğduğu durumdur.

> 📝 ***Matematik Sorusunda Anahtar (Maksimum İhtiyaç):*** *Hoca sayısal Banka algoritması sorusu sorarsa kullanacağın temel formül:* ***İhtiyaç (Need) = Maksimum Kapasite (Max) - Şu An Elinde Olan (Allocation)***

---

6. Deadlock Kurtarma (Recovery) Yöntemleri

Sistem düğümlenmişse bunu nasıl çözeriz? İki ana yöntem vardır:

1. **Process Termination (Görevleri Öldürme):**
   - *Tümünü öldür:* Zaten kilitlenmiş tüm prosesleri iptal et. (Çok maliyetlidir).

- *Tek tek öldür:* Kilitlenme çözülene kadar seçilen bir "kurban" (victim) prosesi öldür. Kurban seçerken prosesin ne kadar süredir çalıştığına, önceliğine bakılır.

2. **Resource Preemption (Kaynağı Zorla Geri Alma):** Kilitli prosesin elindekini zorla alıp başkasına vermek. Sistemde prosesi biraz geriye sarmayı (Rollback) gerektirir.

🎯 SINAV PROVASI: BOŞLUK DOLDURMA VE KLASİK SORULAR

1. Deadlock oluşması için dört şartın aynı anda oluşması gerekir. Bunlar: Karşılıklı Dışlama, Tut ve Bekle, **[Döngüsel Bekleme (Circular Wait)]** ve **[Zorla Alamama (No Preemption)]** şartlarıdır.
2. İşletim sisteminde kilitlenmeleri önceden sezerek onlardan uzak durmaya çalışan, kaynakların tahsisinin güvenli (safe) olup olmadığını kontrol eden klasik algoritmaya **[Banker Algoritması (Banker's Algorithm)]** denir.
3. Çoğu modern işletim sistemi (Windows, Unix vb.), kilitlenmelerin tespiti ve önlenmesinin aşırı performans kaybı yaratması sebebiyle problemi tamamen görmezden gelen **[Devekuşu (Ostrich)]** algoritmasını kullanır.
4. Kaynak tahsis grafiğinde, eğer her kaynağın yalnızca bir kopyası varsa ve grafikte bir **[Döngü / Cycle]** oluşmuşsa bu durum %100 deadlock olduğunu garantiler.

❓ KLASİK / TEST SORUSU POTANSİYELLERİ:

Soru 1 (Senaryo Yorumlama): *Safe State (Güvenli Durum) ile Deadlock arasında nasıl bir ilişki vardır? Güvensiz (Unsafe) duruma geçen sistem kesinlikle kilitlenmiş midir?* **Cevap:** Safe state içerisinde bulunan bir sistemde **asla** deadlock yaşanmaz. Çünkü sistemi sorunsuz bitirecek bir çalışma sırası (safe sequence) bilinmektedir. Ancak sistem Unsafe (Güvensiz) duruma düşerse, bu *kesinlikle kilitleneceği* anlamına GELEMEZ. Sadece kilitlenme (deadlock) ihtimalinin olduğu, riskli bölgeye girdiği anlamına gelir.

Soru 2 (Karşılaştırma): *"Deadlock Önleme (Prevention)" ile "Deadlock Kaçınma (Avoidance)" arasındaki temel felsefe farkı nedir?* **Cevap:** Önleme (Prevention) yönteminde, kilitlenmeye sebep olan 4 fiziksel şarttan en az birinin asla gerçekleşmeyeceğini donanım ve yazılım kısıtlamalarıyla (katı kurallarla) garanti altına alırız (Örn: hiçbir zaman proses kaynak tutarken beklemesin). Kaçınma (Avoidance) yönteminde ise kurallar esnektir, ancak sistem arka planda bir muhasebeci (Banker Algoritması) gibi hesap yapar. Bir kaynak istendiğinde, "Eğer bunu verirsem sistem güvensiz duruma düşer mi?" hesaplaması yaparak sadece güvenli olduğu zamanlarda izin verir. Taleplere dinamik cevap üretir.